

Autonomous Short-Form Video Pipeline Specification

Zero-marginal-cost content engine for autonomous YouTube Shorts & Instagram Reels.

1. Core System Architecture

Orchestrates local/free LLMs, MoneyPrinterTurbo (FastAPI/FFmpeg), Edge-TTS, and official social media publishing APIs.

```
[Scheduler] -> [1. Script Generation (Ollama / Groq Free)]
              -> [2. Video Rendering Engine (MoneyPrinterTurbo :8080)]
                  - Microsoft Edge-TTS (Zero-cost neural voices)
                  - Pexels API (Stock video footage)
                  - FFmpeg (9:16 vertical render & subtitle burning)
              -> [3. Temporary Cloud Staging (Cloudflare R2 / S3 / Catbox)]
              -> [4a. YouTube Data API v3 (Shorts Direct Resumable Upload)]
              -> [4b. Meta Graph API (Reels Container & Publish)]
```

2. Free & Unlimited LLM Engine Selection (\$0.00)

Engine	Cost	Quota	Protocol	Primary Advantage
Ollama (Self-Hosted)	\$0.00	Unlimited	HTTP :11434	Zero rate limits, runs locally on host hardware.
Groq Cloud	\$0.00	14,400 RPD	OpenAI REST	500+ tokens/sec, no local compute required.
Google Gemini Free	\$0.00	1,500 RPD	Google REST	High reasoning, strict 15 RPM throttling.

3. Headless MoneyPrinterTurbo API Configuration (config.toml)

```
[app]
listen_host = "0.0.0.0"
listen_port = 8080
endpoint = "http://127.0.0.1:8080"
llm_provider = "ollama"
ollama_base_url = "http://127.0.0.1:11434"
ollama_model_name = "qwen2.5:7b"
video_source = "pexels"
pexels_api_keys = ["YOUR_PEXELS_KEY"]
voice_name = "en-US-ChristopherNeural"
subtitle_provider = "edge"
```

4. Master Orchestrator Architecture (pipeline_runner.py & orchestrator.py)

The pipeline unifies YouTube Shorts and Instagram Reels publishing into a single autonomous flow:

```
import os, time, json, requests
from src.content.generator import generator
from src.content.mpt_client import mpt_client
from src.storage.uploader import uploader
from src.instagram.publisher import publisher
from src.youtube.shortcuts_publisher import youtube_publisher

def execute_autonomous_run(topic):
    # 1. Script generation via Ollama / Groq fallback
    meta = generator.generate_reel_script(topic, "AI Tool Breakdown")

    # 2. Render 9:16 vertical video via local MoneyPrinterTurbo
    dest_path = f"output/reel_{int(time.time())}.mp4"
    video_file = mpt_client.render_reel(script=meta["narration"], subject=meta["subject"], dest=dest_path)

    # 3. Stage video to S3/R2/Catbox (Public HTTPS URL required by Meta Graph API)
    public_url = uploader.upload_video_file(video_file, "today")

    # 4. Upload to YouTube Shorts via YouTube Data API v3
    yt_id = youtube_publisher.upload_short(
        video_path=video_file,
        title=meta.get("title", f"{topic} #Shorts"),
        description=meta.get("caption", ""),
        tags=meta.get("hashtags", [])
    )

    # 5. Publish to Instagram Reels via Meta Graph API
    ig_media_id = publisher.publish_reel(public_url, meta["caption"])
    return {"youtube_id": yt_id, "instagram_media_id": ig_media_id}
```

5. Production Constraints & Operational Guardrails

- **Rendering Load:** Limit FFmpeg rendering to 2 concurrent tasks per 8 CPU cores to prevent out-of-memory crashes.
- **YouTube API Quota:** 10,000 units/day default allocation (~1,600 units per upload = maximum 6 autonomous uploads daily).
- **Instagram Reels Transcoding:** Container status polling requires up to 180s for Meta video ingestion before publishing.
- **Disk Retention:** Daily cleanup purges rendered MP4s older than 48 hours to preserve SSD headroom.